

■ Security Analysis Report

Generated: 2026-02-13 15:01:41

Summary

Metric	Value
Total Vulnerabilities	16
Security Score	23/100
Critical	3
High	7
Medium	6
Low	0

Vulnerabilities

[1] Taint Analysis: SQL Injection

Severity	Critical
Category	Data Flow Vulnerability
File	test_samples\vulnerable.php
Line	7
Rule ID	TAINT-SQL-INJECTION

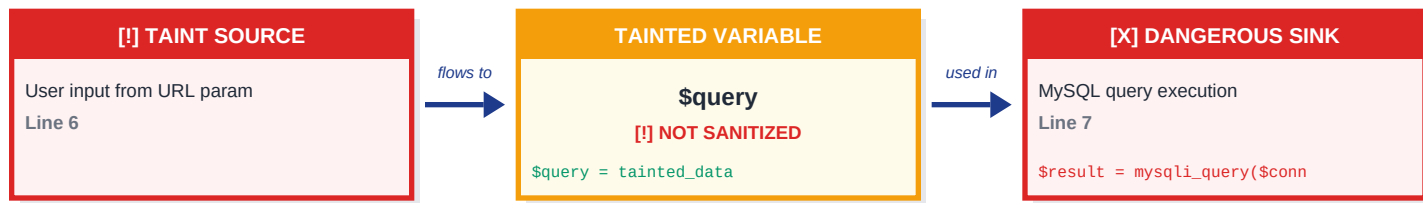
Description: Tainted data from User input from URL parameters (line 6) flows through variable 'query' to MySQL query execution

Code:

```
$query = "SELECT * FROM users WHERE id = " . $userId; $result = mysqli_query($conn, $query); // A03: XSS
```

Remediation: Sanitize data from 'User input from URL parameters' before using in 'MySQL query execution'. Data flow: Source: User input from URL parameters -> Propagated to query

TAINT DATA FLOW ANALYSIS



[!] SECURITY VULNERABILITY: TAIN'TED DATA REACHES SINK WITHOUT SANITIZATION [!]

File: test_samples\vulnerable.php

[2] Taint Analysis: Command Injection

Severity	Critical
Category	Data Flow Vulnerability
File	test_samples\vulnerable.php
Line	23
Rule ID	TAINT-COMMAND-INJECTION

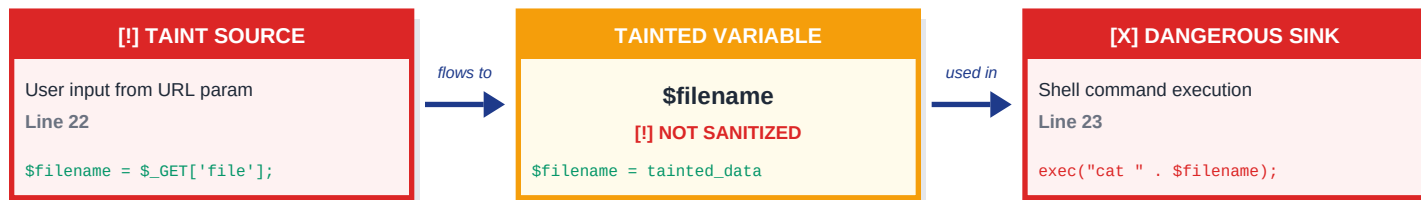
Description: Tainted data from User input from URL parameters (line 22) flows through variable 'filename' to Shell command execution

Code:

```
$filename = $_GET['file']; exec("cat " . $filename); // A10: SSRF
```

Remediation: Sanitize data from 'User input from URL parameters' before using in 'Shell command execution'. Data flow: Source: User input from URL parameters

TAINT DATA FLOW ANALYSIS



[!] SECURITY VULNERABILITY: TAIN'TED DATA REACHES SINK WITHOUT SANITIZATION [!]

File: test_samples\vulnerable.php

[3] Taint Analysis: Path Traversal

Severity	High
Category	Data Flow Vulnerability
File	test_samples\vulnerable.php
Line	27
Rule ID	TAINT-PATH-TRAVERSAL

Description: Tainted data from User input from URL parameters (line 26) flows through variable 'url' to File system operation

Code:

```
$url = $_GET['url']; $content = file_get_contents($url); // A07: Weak Session Management
```

Remediation: Sanitize data from 'User input from URL parameters' before using in 'File system operation'. Data flow: Source: User input from URL parameters

TAINT DATA FLOW ANALYSIS



[!] SECURITY VULNERABILITY: TAIN'TED DATA REACHES SINK WITHOUT SANITIZATION [!]

File: test_samples\vulnerable.php

[4] Taint Analysis: Server-Side Request Forgery

Severity	High
Category	Data Flow Vulnerability
File	test_samples\vulnerable.php
Line	27
Rule ID	TAINT-SERVER-SIDE-REQUEST-FORGERY

Description: Tainted data from User input from URL parameters (line 26) flows through variable 'url' to Remote file access

Code:

```
$url = $_GET['url']; $content = file_get_contents($url); // A07: Weak Session Management
```

Remediation: Sanitize data from 'User input from URL parameters' before using in 'Remote file access'. Data flow: Source: User input from URL parameters

TAINT DATA FLOW ANALYSIS



[!] SECURITY VULNERABILITY: TAINED DATA REACHES SINK WITHOUT SANITIZATION [!]

File: test_samples\vulnerable.php

[5] Taint Analysis: Path Traversal

Severity	High
Category	Data Flow Vulnerability
File	test_samples\vulnerable.php
Line	27
Rule ID	TAINT-PATH-TRAVERSAL

Description: Tainted data from User input from URL parameters (line 26) flows through variable 'url' to Remote file access

Code:

```
$url = $_GET['url']; $content = file_get_contents($url); // A07: Weak Session Management
```

Remediation: Sanitize data from 'User input from URL parameters' before using in 'Remote file access'. Data flow: Source: User input from URL parameters

TAINT DATA FLOW ANALYSIS



[!] SECURITY VULNERABILITY: TAINED DATA REACHES SINK WITHOUT SANITIZATION [!]

File: test_samples\vulnerable.php

[6] Missing Input Validation

Severity	Medium
Category	Best Practice
File	test_samples\vulnerable.php
Line	5
Rule ID	BP-PHP-001

Description: Direct use of superglobals without validation

Code:

```
// A03: SQL Injection $userId = $_GET['id']; $query = "SELECT * FROM users WHERE id = "
. $userId; $result = mysqli_query($conn, $query);
```

Remediation: Validate and sanitize all input using `filter_input()` or `filter_var()`

[7] Missing Input Validation

Severity	Medium
Category	Best Practice
File	test_samples\vulnerable.php
Line	10
Rule ID	BP-PHP-001

Description: Direct use of superglobals without validation

Code:

```
// A03: XSS echo $_GET['name']; print $_POST['comment'];
```

Remediation: Validate and sanitize all input using filter_input() or filter_var()

[8] Missing Input Validation

Severity	Medium
Category	Best Practice
File	test_samples\vulnerable.php
Line	11
Rule ID	BP-PHP-001

Description: Direct use of superglobals without validation

Code:

```
echo $_GET['name']; print $_POST['comment']; // A02: Weak Password Hashing
```

Remediation: Validate and sanitize all input using `filter_input()` or `filter_var()`

[9] Missing Input Validation

Severity	Medium
Category	Best Practice
File	test_samples\vulnerable.php
Line	14
Rule ID	BP-PHP-001

Description: Direct use of superglobals without validation

Code:

```
// A02: Weak Password Hashing $password = $_POST['password']; $hash = md5($password);
```

Remediation: Validate and sanitize all input using filter_input() or filter_var()

[10] Weak Password Hashing

Severity	Critical
Category	A02
File	test_samples\vulnerable.php
Line	15
Rule ID	A02-PHP-001

Description: Use of weak cryptographic hash functions

Code:

```
$password = $_POST['password']; $hash = md5($password); // A05: Hardcoded Secrets
```

Remediation: Use password_hash() with PASSWORD_BCRYPT or PASSWORD_ARGON2ID

[11] Hardcoded Secrets

Severity	High
Category	A05
File	test_samples\vulnerable.php
Line	18
Rule ID	A05-PHP-001

Description: Hardcoded credentials or secrets in source code

Code:

```
// A05: Hardcoded Secrets $password = "admin123"; $api_key =  
"sk_live_1234567890abcdef";
```

Remediation: Use environment variables or secure configuration files outside web root

[12] Hardcoded Secrets

Severity	High
Category	A05
File	test_samples\vulnerable.php
Line	19
Rule ID	A05-PHP-001

Description: Hardcoded credentials or secrets in source code

Code:

```
$password = "admin123"; $api_key = "sk_live_1234567890abcdef"; // A03: Command  
Injection
```

Remediation: Use environment variables or secure configuration files outside web root

[13] Missing Input Validation

Severity	Medium
Category	Best Practice
File	test_samples\vulnerable.php
Line	22
Rule ID	BP-PHP-001

Description: Direct use of superglobals without validation

Code:

```
// A03: Command Injection $filename = $_GET['file']; exec("cat " . $filename);
```

Remediation: Validate and sanitize all input using filter_input() or filter_var()

[14] Missing Input Validation

Severity	Medium
Category	Best Practice
File	test_samples\vulnerable.php
Line	26
Rule ID	BP-PHP-001

Description: Direct use of superglobals without validation

Code:

```
// A10: SSRF $url = $_GET['url']; $content = file_get_contents($url);
```

Remediation: Validate and sanitize all input using `filter_input()` or `filter_var()`

[15] Server-Side Request Forgery

Severity	High
Category	A10
File	test_samples\vulnerable.php
Line	27
Rule ID	A10-PHP-001

Description: File/URL access with user-controlled input, vulnerable to SSRF

Code:

```
$url = $_GET['url']; $content = file_get_contents($url); // A07: Weak Session Management
```

Remediation: Validate URLs, whitelist allowed domains, block internal network access

[16] Weak Session Management

Severity	High
Category	A07
File	test_samples\vulnerable.php
Line	30
Rule ID	A07-PHP-001

Description: Missing session security configuration

Code:

```
// A07: Weak Session Management session_start(); // Missing session security  
configuration
```

Remediation: Configure secure session settings: httpOnly, secure flag, SameSite attribute, regenerate session ID on login